

HS Information Retrieval
WS 03/04
PD Dr. Karin Haenelt
Lehrstuhl für Computerlinguistik
Universität Heidelberg

Stemming

Methoden und Ergebnisse

Dagmar Karlson
Alphonsine Kouadio
Ineta Sējāne
Dirk Weissmann

Themen

Einführung und Definition

Methoden des Stemming

Evaluation von Stemming und einzelnen
Stemmern

Porter Stemmer für 3 Sprachen

Nice Stemmer

Definition Stemming

Als Stemming bezeichnet man ein Verfahren zum Zusammenführen von lexikalisch verwandten Termen. Dies wird durch Conflation realisiert.

Conflation

Conflation – die Abbildung mehrerer morphologischer Varianten eines Wortes auf eine Repräsentation – stem bzw. Stamm.

Idee:

Der Stamm trägt die Bedeutung des Konzeptes, das mit diesem Wort verbunden ist; Affixe modifizieren das Konzept oder tragen syntaktische Informationen bei.

!!! Ein Stamm in der Grammatik ist oft nicht gleich mit dem Stamm im Stemming, z.B.:

beauty (Grammatik) vs. beauti (Stemming)

Beispiele

calculat

calculate, calculates, calculation, calculations, calculating

comput

compute, computes, computed, computing, computable,
computability, computation, computational, computer, computerese,
computerize

!!! Eigennamen und Akronyme sollten nicht gestemmt werden,
werden aber i.d.R. mitgestemmt und ergeben Fehler:

Miller Miller
 *mill

In manchen Texten machen Eigennamen bis zu 15% aller Wörter
aus, z.B. in TREC-Testkorpora.

Stemming im Information Retrieval

Schnittstellen, gängige Anordnung:

Tokenizer **Stemmer** Indexer

Stoppwörter werden aussortiert

Korpus wird gestemmt

doppelte Stämme werden aussortiert

aus gestemmtten Wörtern werden Indexterme
erstellt

Stemming im Information Retrieval

Stemmen erfolgt entweder

- 1) zur Indexierungszeit oder
- 2) zur Suchzeit.

Zu 1):

Vorteile: Effizienz und Verkleinerung der Indexdateien, weniger Arbeit zur Suchzeit

Nachteile: Die Information über den ursprünglichen Term geht verloren oder man braucht zusätzlichen Speicher, wo beides – gestemmte und nicht gestemmte Formen – abgespeichert werden.

Zu 2):

Nachteil: Mehr Arbeit zur Suchzeit

Precision vs. Recall

Stemming ist eine Entscheidung:

precision vs. recall

Stemming Algorithmen verbessern den recall fast immer und verschlechtern in der Regel die precision.

$$\text{Precision} = \frac{\text{gefundene relevante Dokumente}}{\text{alle gefundenen Dokumente}}$$

$$\text{Recall} = \frac{\text{gefundene relevante Dokumente}}{\text{alle relevanten Dokumente}}$$

Ein Überblick über die verschiedenen Arten der Stemmer

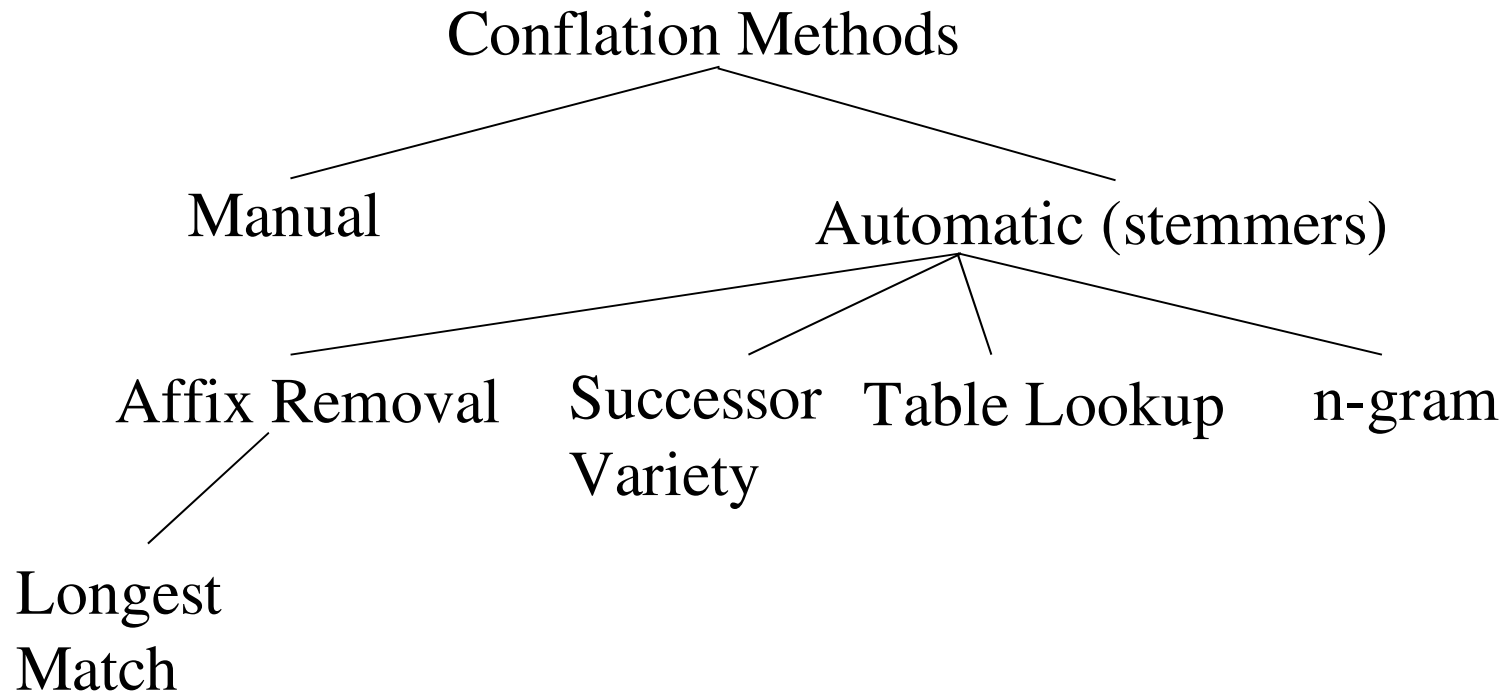
Manuell

Dictionary- und Table look-up Stemmers

Successor Stemmers

N-grams

Affix removal Stemmers (AR-Stemmer)



Manuelle Conflation

Mit Hilfe regulärer Ausdrücke

z.B. run*

run, runs, running, runnings, runner, runners, runny

aber auch z.B.:

rung, runabout, runaway, runway, runtime,

runt, rune,

Dictionary Look-Up Stemmers

Dictionary Stemmer sind Look-Up-Stemmer und in der Funktion bei den Table-Look-Up Stemmern (s. Übersicht Folie 10) einzuordnen.

- einige wenige sehr einfache und möglichst zuverlässige Regeln, z.B. Pluralbildung
- Das neu gewonnene Wort wird mit einem Wörterbuch verglichen und durch den Stamm ersetzt, der ihm [am meisten] gleicht.

Bekannte Systeme: INQUERY und RetrievalWare Systems

Stemming in Retrieval System

INQUERY

INQUERY nutzt die Kstem-Technik. Kstem verschmelzt (alle) morphologischen Varianten eines Wortes zu seinem Stamm (conflation).

Generell reduziert Kstem nur auf Wörter, die im Wörterbuch stehen. Nur wenige Suffixe werden entfernt, wenn das neu gewonnene Wort nicht im Wörterbuch steht, wie z.B. -ness und -ly.

Wörter, die in der Form, wie sie eingegeben wurden, im Wörterbuch stehen, werden nicht gestemmt, da man annimmt, dass sie nicht weiter zurück ableitbar sind, z.B. wenn *factorial* im Wörterbuch steht, wird es nicht zu *factory* reduziert.

Es versucht, nicht zu weit zu reduzieren, um Konflikte zu vermeiden, z.B. *memorial* und *memorize* werden nicht zu *memory* reduziert.

Für nicht reguläre Ableitungen wird der gewünschte Stamm explizit angegeben wie z.B. *matrices* *matrix*

Stemming im Retrieval- System RetrievalWare

Dieses System hat einen Thesaurus/semantic network mit über 400.000 Wörtern als integrierte Datenstruktur. Die Wörter sind mit ihren morphologischen Varianten darin enthalten.

Dem Thesaurus unbekannte Wörter werden nach morphologischen Regeln gestemmt.

Vor- und Nachteile von Look-Up-Verfahren

Vorteile:

Stemming-Algorithmen sind mit einem Wörterbuch verbunden, so wird die precision gewährleistet.

Nachteile:

Tabellen-look-up erfordert eine große zusätzliche Datenstruktur ↑ Speicherplatz.

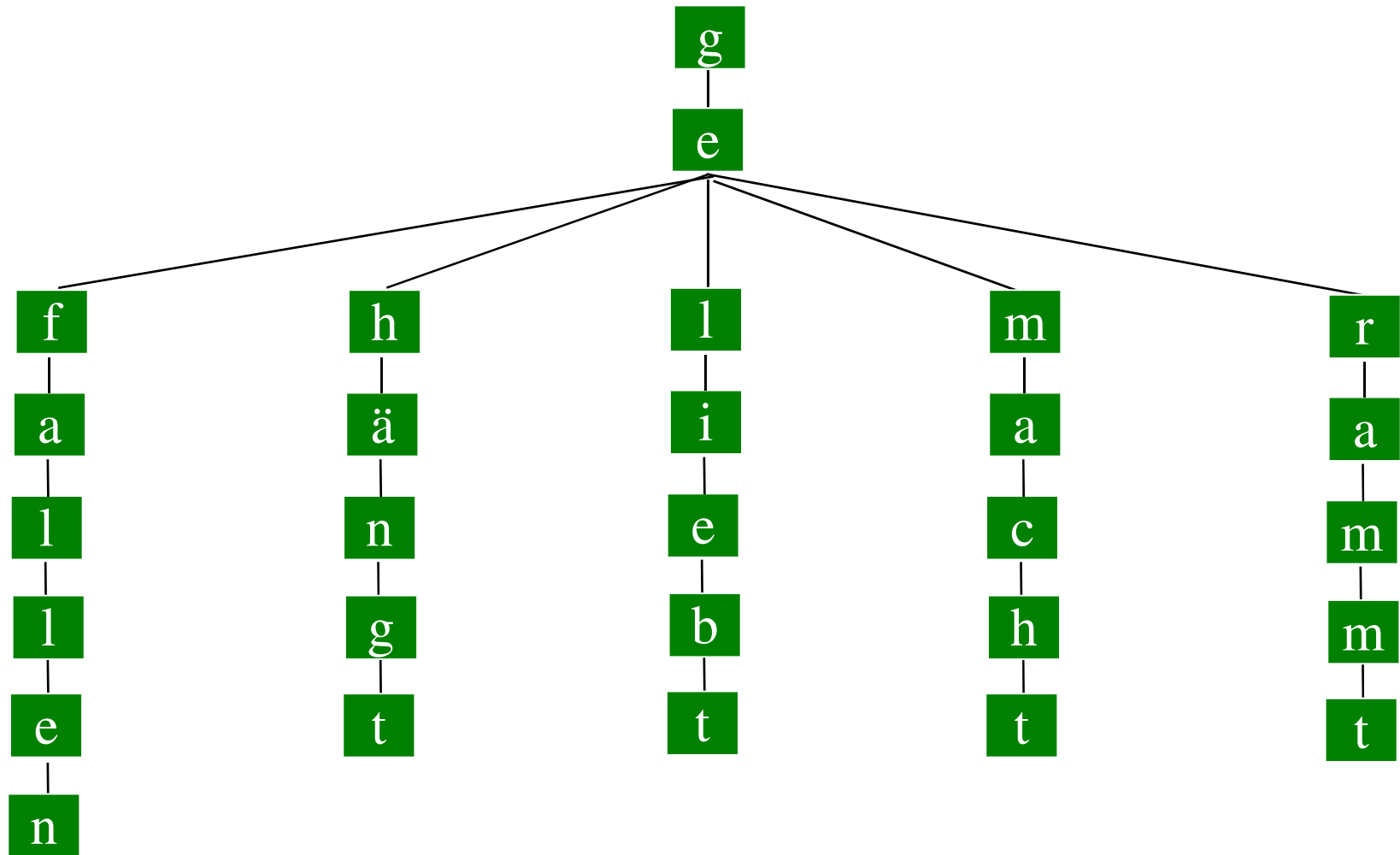
Der Aufbau des Systems ist zeit- und arbeitsintensiv und bedarf regelmäßiger Pflege.

Successor Stemming

Nachfolger werden überprüft

Wichtig: Nachfolgervielfalt

Ein Baum für die Terme geliebt, gefallen, gehängt, gemacht, gerammt



Successor Stemming: Methoden

Cutoff

Peak and Plateau

Complete Word

Entropy

Successor Stemming:

Entropie - Methode: Indikatoren

α : Wort

n : Wortlänge

α_i = Präfix von α der Länge i

D = Textkorpus

D_{α_i} = Untermenge von D , die sämtliche Terme enthält, deren erste i Buchstaben mit α_i exakt übereinstimmen

S_{α_i} = Nachfolgervielfalt von α_i

Successor Stemming: Entropie-Methode

$|D_{\alpha i}|$ = Menge der Wörter mit der Sequenz der Buchstaben in α der Länge i

$|D_{\alpha ij}|$ = Menge der Wörter in $D_{\alpha i}$ mit dem Nachfolger j

$|D_{\alpha ij}| / |D_{\alpha i}|$ = Wahrscheinlichkeit der Nachfolge j von α

Entropie: $H_{\alpha i} = \sum_{j=1}^{26} - |D_{\alpha ij}| / |D_{\alpha i}| \cdot \lg |D_{\alpha ij}| / |D_{\alpha i}|$

n-gram - Methode

Digramme erstellen

Anhand von einmaligen Digrammen innerhalb eines Terms, aber nicht innerhalb des gesamten Korpus, wird ein Ähnlichkeitsmaß berechnet.

n-gram: Beispiel für Digramme

„lieben“ und „geliebt“

li ie eb be en (5)

ge el li ie eb bt (6)

li ie eb (3)

n-gram – Methode: Ähnlichkeitsberechnung

A = einmalige Digramme im 1. Wort

B = einmalige Digramme im 2. Wort

C = einmalige Digramme in beiden Wörtern
vertreten

$$S = 2C / (A+B)$$

Beispiel: $S = 6 / 11 = 0,5$

Aus Tests geht hervor, dass mit einem Ergebnis von 0,6 zehn von elf Clustern korrekt zusammengefasst werden.

Stemming durch Entfernen von Affixen

Es ist die am weitesten verbreitete Art von Stemmern.

Die meisten Stemmer sind iterativ und versuchen entweder die längstmöglichen Affixe zu entfernen (z.B. Lovins-68, Salton-68, Dawson-74, Porter-80, Paice-90) oder stemmen flach, indem sie nur bestimmte Flexionen entfernen (z.B. Simple Stemmer, Inflection Stemmer).

Auf Porter-, Simple- und Inflection-Stemmer gehen wir nach der allgemeinen Einführung detailliert ein.

Evaluation von Stemmern

- Korrektheit
 - overstemming – zu viel entfernt, nicht relevante Dokumente ausgegeben
 - understemming - zu wenig entfernt, nicht alle relevanten Dokumente ausgegeben
- Effektivität
 - recall, precision, Geschwindigkeit
- Kompressionsrate
 - Größe der Indexdateien

Die linguistische Korrektheit ist nicht wichtig.

Evaluation und Ergebnisse

Beispiele

Tiefes vs. flaches (= nur *s* abgetrennt) Stemming (Salton, 1986, vgl. Frakes 143f)

Terme insgesamt	tiefes Stemming bevorzugt	s-Stemming bevorzugt	keines bevorzugt	Bemerkungen
476	272	132	72	IRE-3 Informatik abstracts
490	254	107	129	ADI papers
688	134	371	183	Cranfield-1 Aerodynamik abstracts

„For none of the collections (ADI, IRE-3, Cranfield-1) is the improvement of one method over the other really dramatic, so that in practice, either procedure might reasonably be used“ (Frakes, 144).

In Fachtexten mit homogenem Fachvokabular liefern andere Methoden bessere Ergebnisse als in Texten mit gemischtem oder alltäglichem Vokabular (vgl. Frakes, 149).

Evaluation und Ergebnisse

Beispiele

Lennon et al. (1981) vergleicht die Unterschiede in der Effektivität von 8 unterschiedlichen Stemmern gegen nicht gestemnte Terme:

Lovins Stemmer, Porter Stemmer, RADCOL, Suffix Frequency Stemmer, INSPEC, Hafer and Weiss Stemmer (successor Stemmer), Trigram Stemmer, einem Stemmer, der auf der Frequenz der Wortendungen in einem Korpus basiert

Die Methoden wurden mit 2 unterschiedlichen Formeln berechnet:

1) precision war 2-mal wichtiger als recall ($b = 0,5$)

„) recall war 2-mal wichtiger als precision ($b = 2$)

In beiden Berechnungen war die Ausgabe mit gestemnten Termen besser als mit den nicht gestemnten. Eine Ausnahme bildete in beiden Fällen der Hafer and Weiss Stemmer.

„[...] in particular, simple, fully automated methods perform as well as procedures which involve a large degree of manual involvement in their development.“ (Frakes, 145)

Evaluation und Ergebnisse

Beispiele

Frakes (1982) überprüfte die Hypothese, ob automatisches Stemming genauso gute Ergebnisse liefert wie manuelles:

„No significant difference was found between manual and automatic conflation, indicating that conflation can be automated with no significant loss in retrieval performance.“ (Frakes, 147)

Walker & Jones (1987) haben anhand des Porter Algorithmus herausgefunden, dass:

Stemming den recall wesentlich verbessert,
flaches Stemming die precision kaum verschlechtert und
tiefes Stemming die precision deutlich verschlechtert.

Sie empfehlen tiefes Stemming nur, wenn keine Dokumente gefunden werden.

Zusammenfassung Affix Removal-Stemmer

Vorteile:

Schnell, leicht zu implementieren, erfordern keine großen Ressourcen

Nachteile:

Präzisionsverlust, Anomalien (falsch gestemmt Wörter, gestemmt Eigennamen)

Es werden sowohl die Anfrage als auch die zu durchsuchenden Texte gestemmt, deshalb lässt sich die Ausgabe im Falle eines Fehlers nicht ohne Weiteres nachvollziehen.

Porter Stemming Algorithm

Der Porter-Algorithmus ist ein Affix-Removal-Stemmer (s. Übersicht Folie 10)

Der Algorithmus basiert auf einer Reihe von Bedingungen für die Suffixe. Diesen Bedingungen entsprechen Aktionen, die am Wort durchzuführen sind.

Es werden Listen von Konsonanten und Vokalen definiert; v ist ein Einzelvokal, V ist die Liste von Vokalen mit mindestens einem Eintrag, entsprechend c und C für Konsonanten.

Dann werden Sequenzen der Buchstaben herausgearbeitet, z.B. fürs

Englische:
 $C?(VC)\{m\}V?$ ist die Form eines Wortes

C (consonant) und V (vowel) sind die optionalen Anfangsbuchstaben und m die Zahl, wie oft sich die Sequenz VC wiederholt, z.B.

m=0 *free, why*

m=1 *frees, whose*

m=2 *prologue, compute*

Es werden Bedingungen für eine Regel formuliert:

- *<X> Stamm endet mit dem Buchstaben X
- *v* Stamm enthält einen Vokal
- *d Stamm endet mit einem Doppelkonsonanten
- *o Stamm endet mit einer CVC-Sequenz und der letzte Konsonant ist kein w, x, oder y.

Form der Regel ist: (Bedingung) S1 -> S2, wobei S1 und S2 Suffixe eines Wortes sind.

Der Algorithmus aus

<http://snowball.tartarus.org/porter/stemmer.html>

Step 1a

SSES -> SS caresses -> caress

IES -> I ponies -> poni

ties -> ti

SS -> SS caress -> caress

S -> cats -> cat

Step 1b

(m>0) EED -> EE feed -> feed

agreed -> agree

(*v*) ED -> plastered -> plaster

bled -> bled

(*v*) ING -> motoring -> motor

sing -> sing

If the second or third of the rules in Step 1b is successful, the following is done:

AT -> ATE	conflat(ed) -> conflate
BL -> BLE	troubl(ed) -> trouble
IZ -> IZE	siz(ed) -> size
(*d and not (*L or *S or *Z)) -> single letter	hopp(ing) -> hop
	tann(ed) -> tan
	fall(ing) -> fall
	fall(ing) -> fall
	hiss(ing) -> hiss
	fizz(ed) -> fizz
(m=1 and *o) -> E	fail(ing) -> fail
	fil(ing) -> file

The rule to map to a single letter causes the removal of one of the double letter pair. The -E is put back on -AT, -BL and -IZ, so that the suffixes -ATE, -BLE and -IZE can be recognised later. This E may be removed in step 4.

Step 1c

(*v*) Y -> I	happy -> happi
	sky -> sky

Step 1 deals with plurals and past participles. The subsequent steps are much more straightforward.

Step 2

(m>0) ATIONAL -> ATE	relational -> relate
(m>0) TIONAL -> TION	conditional -> condition
	rational -> rational
(m>0) ENCI -> ENCE	valenci -> valence
(m>0) ANCI -> ANCE	hesitanci -> hesitance
(m>0) IZER -> IZE	digitizer -> digitize
(m>0) ABLI -> ABLE	conformabli -> conformable
(m>0) ALLI -> AL	radicalli -> radical
(m>0) ENTLI -> ENT	differentli -> different
(m>0) ELI -> E	vileli -> vile
(m>0) OUSLI -> OUS	analogousli -> analogous
(m>0) IZATION -> IZE	vietnamization -> vietnamize
(m>0) ATION -> ATE	predication -> predicate
(m>0) ATOR -> ATE	operator -> operate
(m>0) ALISM -> AL	feudalism -> feudal
(m>0) IVENESS -> IVE	decisiveness -> decisive
(m>0) FULNESS -> FUL	hopefulness -> hopeful
(m>0) OUSNESS -> OUS	callousness -> callous
(m>0) ALITI -> AL	formaliti -> formal
(m>0) IVITI -> IVE	sensitiviti -> sensitive
(m>0) BILITI -> BLE	sensibiliti -> sensible

The test for the string S1 can be made fast by doing a program switch on the penultimate letter of the word being tested. This gives a fairly even breakdown of the possible values of the string S1.

It will be seen in fact that the S1-strings in step 2 are presented here in the alphabetical order of their penultimate letter. Similar techniques may be applied in the other steps.

Step 3

(m>0) ICATE -> IC	triplicate -> triplic
(m>0) ATIVE ->	formative -> form
(m>0) ALIZE -> AL	formalize -> formal
(m>0) ICITI -> IC	electriciti -> electric
(m>0) ICAL -> IC	electrical -> electric
(m>0) FUL ->	hopeful -> hope
(m>0) NESS ->	goodness -> good

Step 4

(m>1) AL ->	revival -> reviv
(m>1) ANCE ->	allowance -> allow
(m>1) ENCE ->	inference -> infer
(m>1) ER ->	airliner -> airlin
(m>1) IC ->	gyroscopic -> gyroscop
(m>1) ABLE ->	adjustable -> adjust

(m>1) IBLE ->	defensible -> defens
(m>1) ANT ->	irritant -> irrit
(m>1) EMENT ->	replacement -> replac
(m>1) MENT ->	adjustment -> adjust
(m>1) ENT ->	dependent -> depend
(m>1) and (*S or *T)) ION ->	adoption -> adopt
(m>1) OU ->	homologou -> homolog
(m>1) ISM ->	communism -> commun
(m>1) ATE ->	activate -> activ
(m>1) ITI ->	angulariti -> angular
(m>1) OUS ->	homologous -> homolog
(m>1) IVE ->	effective -> effect
(m>1) IZE ->	bowdlerize -> bowdler

The suffixes are now removed. All that remains is a little tidying up.

Step 5a

(m>1) E ->	probate -> probat
	rate -> rate
(m=1 and not *o) E ->	cease -> ceas

Step 5b

(m > 1 and *d and *L) -> single letter	controll -> control
	roll -> roll

Nach Kowalski und Maybury (2000) sollten am Wort *duplicatable* folgende Schritte durchgeführt werden:

duplicatable	Eingabe
duplicat	Regel 4
duplicate	Regel 1b1
duplic	Regel 3

Die Regel zum Entfernen von –ic aus dem Schritt 4 wird allerdings nicht mehr durchgeführt, da nach Algorithmus nur eine Regel aus jedem Schritt erlaubt ist.

!!! Der Original-Porter aus dem Internet auf der snowball.tartarus.org-Seite leistet das nicht, er stemmt sequentiell. Ein Zurückspringen aus der Regel 4 in Regel 1b1 ist nicht möglich.

duplicatable -> duplicat (Regel 4)

Weitere Regeln kommen hier nicht zur Anwendung.

Test von Wirtschaftstexten

Es wurde eine größere Anzahl von englischen Texten gestemmt. Um die Ausgabe evaluieren zu können, wurde zusätzlich zum Algorithmus eine verkettete Liste implementiert, die alphabetisch sortiert ist. In ihr sind gleiche Wörter zusammengefasst, deren Häufigkeit im Text angegeben ist. Parallel dazu werden die gestemmtten Wörter ausgegeben. Die Ausgabe erfolgt wahlweise in eine Datei oder auf die Standardausgabe.

Ergebnisse

Ein Korpus aus diversen englischen Texten von insgesamt **62.399** Wörtern wird zuerst gegen eine Stoppwörterliste (541 Wörter) abgeglichen. 39.358 Stoppwörter (63,07%) werden aussortiert. Nach dem Zusammenfassen gleicher Wörter bleiben 5.502 Einträge. Nach dem Stemmen werden diese Einträge auf **4.442** Stämme, also um ca. 80,73% reduziert. Aus dem Korpus verbleiben 7,11% als Indexterme.

Verben

Bei regelmäßigen Verben kommt es durch den Porter-Algorithmus kaum zu Problemen.

Beispiele:

include	includ
included	includ
includes	includ
including	includ

Gegenbeispiele:

exceed	exce
exceeded	exceed
exceeding	exceed
exceeds	exce

Verben und Substantive

Relativ selten werden Verben und Substantive auf den selben Stamm konflationiert.

Beispiel:

expect	expect
expectation	expect
expectations	expect
expected	expect
expecting	expect
expects	expect

Gegenbeispiel:

gain	gain
gained	gain
gainers	gainer
gaining	gain
gains	gain

Probleme

Der Porter - Algorithmus konflationiert Wörter, die auf die Endung -er enden, nicht. Damit soll vermieden werden, dass Personennamen wie z. B. Porter auf ihren Stamm reduziert werden. Dies resultiert in der Nichtbehandlung von Nomen, die aus dem Anhängen von -er an das Verb entstanden sind.

read(er), paint(er), usw.

Außerdem wird der Komparativ nicht auf seinen Stamm reduziert.

quick(er), busi(er), usw.

Ein weiteres Problem ergibt sich durch die Abbildung von nicht verwandten oder zu weit verwandten Wörtern auf den selben Stamm.

Beispiele:

communications commun

community commun

compete compet

competence compet

competency compet

competent compet

competing compet

designated design

designed design

designs design

Porter fürs Französische

Anmerkungen zur Implementierung

verfügbare Version:

<http://snowball.tartarus.org/french/stemmer.html>

- implementiert in Snowball
- automatisch übersetzt in C
- C-Quelle ließ sich mit cc- und gcc-Compiler nicht kompilieren

unsere Alternative:

- Verwendung der Version für das Englische
- Anpassung für das Französische

Anpassungen:

- zusätzliche Funktionen für diakritische Zeichen
- die zusätzlich implementierte Liste (s. auch Englisch)
- einige Regeln wurden abgeändert

Porter fürs Französische

Unterschiede zum Englischen

Es gibt 6 Schritte mit Unterschriften im Unterschied zum Englischen mit
5

Schritten, weil im Französischen die Suffixzusammensetzungen
reichhaltiger sind:

Der französische Algorithmus auf der Internetseite <http://snowball.tartarus.org/french/stemmer.html>

unterteilt die Wörter zunächst in Regionen:

„If the word begins with two vowels, *RV* is the region after the third letter, otherwise the region after the first vowel not at the beginning of the word, or the end of the word if these positions cannot be found.

a i m e r a d o r e r
|...| |.....|

R1 is the region after the first non-vowel following a vowel, or the end of the word if there is no such non-vowel. *R2* is the region after the first non-vowel following a vowel in *R1*, or the end of the word if there is no such non-vowel.

f a m e u s e m e n t

|.....R1....|

|...R2...| „

Da wir den englischen Porter-Algorithmus verwendet haben, haben wir zur Umsetzung der Regionen die im Porter schon vorhandene Funktion $m()$ benutzt ($m()$ vergleiche Folie 31).

Für RV haben wir $m = 0$ eingesetzt, für R1 $m > 0$ und für R2 $m > 1$.

Des Weiteren wird im französischen Algorithmus im Gegensatz zum englischen das vorangehende Suffix in die Betrachtung einbezogen.

Beispiel:

Folgt *ement(s)* vor *abl* oder *iqU*, wird *ement(s)* abgetrennt, aber nur wenn

ement(s) in R2 ist, bzw. wenn zuvor *ièr* oder *Ièr* auftritt und *ement(s)* in RV vorkommt.

Im englischen Porter-Algorithmus werden Vorgänger nicht betrachtet. Wenn z. B. *-ment* als Endung abgetrennt werden soll, wird es bei Vorhandensein ohne Berücksichtigung auf seine Vorgänger entfernt.

Zusätzlich werden im französischen Algorithmus im Unterschied zum englischen die Buchstaben:

u sowie *i* , wenn vor und nach ihnen ein Vokal steht,

y vor oder nach einem Vokal und

u nach *q* in Großbuchstaben umgewandelt.

Sie werden aus phonetischen Gründen wie Konsonanten behandelt.

Beispiel:

ennuie -> *ennule* *inquiétude* -> *inqUiétude*

Dies geschieht, um im Einzelfall zu unterscheiden, welche Buchstabenfolgen reduziert werden können: In *ennule* wird nichts abgehängt, während in *mairie* *ie* abgehängt wird._

Nach dem Stemmen belassen wir die Großbuchstaben in ihrem Zustand und wandeln sie nicht wieder in Kleinbuchstaben um. Somit wird dem Betrachter ersichtlich, wann z. B. *ie* abgetrennt wurde.

Analyse und Ergebnisse

Ein Korpus aus diversen französischen Texten von insgesamt 66.980 Wörtern wird zuerst gegen eine Stoppwörterliste (171 Wörter) abgeglichen. 32.369 Stoppwörter (48,33%) werden aussortiert. Nach dem Zusammenfassen gleicher Wörter bleiben 6.912 Einträge. Nach dem Stemmen werden diese Einträge auf 5.428 Stämme, also um ca. 78,53 % reduziert. Aus dem Korpus verbleiben 8,1% als Indexterme.

Unsere Originalentwicklung: Der Porter Algorithmus fürs Lettische

Endungen

Deklination (7 Fälle, ähnlich wie das Latein)

Konjugation (ähnlich wie das Deutsche)

mehrere Arten Partizipien

Zahlreiche Präfixe

Derivationssuffixe für Nomina, Verben, Adjektive,
Adverbien

Porter-Stemmer

Planung eines Porter-Algorithmus für Lettisch

Anhand des Lexikons und einer Grammatik wurden folgende Fragen überlegt :

Was sind die kleinsten Stämme der Sprache?

Was sind typische Muster der Sprache?

Anhand der Muster wurden Einzelschritte für einen Algorithmus überlegt. Da man pro Schritt immer nur eine Regel realisieren kann, muss besonders auf die Abstimmung der Regeln geachtet werden.

Der gesamte Überlegungsprozess verläuft bekanntlich:

Vom Primitiven über das Komplizierte zum Einfachen.

- **Der kleinste Stamm** im Lettischen besteht aus lediglich einem Diphthong (z.B. *ie-*), die nächsten kleinstmöglichen aus Konsonant + Vokal/Diphthong (*sē-*, *rī-*, *plau-*) und Vokal/Diphthong + Konsonant (*ej-*, *iev-*). Vor diesen Stämmen können Präfixe stehen. Das längste besteht aus 3 Buchstaben *aiz-* .

Wir haben uns entschieden, dass nur Wörter mit 4 oder mehr Buchstaben gestemmt werden, sonst wird das Wort direkt ausgegeben. So wird gewährleistet, dass das Wort nicht zu weit gestemmt wird. Auf eine Definition von Folgen wie CVC wurde verzichtet.

- Alle **Flexionen** werden abgetrennt.
- Bei den **Derivationssuffixen** musste eine Entscheidung getroffen werden, welche überhaupt sinnvollerweise abgetrennt werden, da aus einigen zu weit entfernte Bedeutungen entstehen (↓ precision), z.B.:

zeme (Planet Erde, Boden) – *zemnieks* (Bauer)

mazs (klein) – *mazulis* (Baby)

likt (stellen, legen) – *liktenis* (Schicksal)

labs (gut) – *labība* (Getreide)

Wir haben uns entschieden, bei Substantiven nur *-šan-* und *-um-* zu entfernen. Bei Adjektiven werden überhaupt keine Suffixe entfernt.

I. Flexion

1) Verb, Partizipien

usī, at, aties, ot, st, oties, usi, ies, īs

2) Adjektive

ajiem, ajai, ajam, ajām, ajās, ajos, ajā, ais, ie, os, o

3) Substantive

um, us, im, īm, iem, is, os, as, ās, ai, es, ēs, ei, em, ēm, am, ām, s, š, a, ā, u, ū, i, ī, e, ē

II. Konjugation, Derivation Verben und Partizipien

iet, sit, uš, oš

III. DELETE *j* in *pj, bj, mj, vj*

IV. Komparativ

āk

V. Derivation

1) Partizipien

dam, oš, uš, am, ām

2) Substantive

šan, um

VI. Bearbeitung des Wortendes

j, t, š, s T, d, z, ž Z, ll, ļļ LL, l, ļ L, n, ņ N, c, č, k C,

nach *o, ā, ē, ī, ū* am Wortende wird ein *T* hinzugefügt

GELÖSTE UND UNGELÖSTE PROBLEME:

Lautwechsel

Im Schritt 6 des Algorithmus wird das Problem von konsonantischem Lautwechsel gelöst durch die Einführung von Großbuchstaben, die mehrere Buchstaben abbilden. Das löst folgende Probleme:

<i>t, s, š</i>	<i>starmetis – starmeši (Scheinwerfer, Pl)</i>
<i>d, z, ž</i>	<i>gardēdis – gardēži (Gourmet, Pl)</i>
<i>n, ŋ</i>	<i>sapnis – sapni (Traum, Pl)</i>
<i>c, č</i>	<i>lācis – lāči (Bär, Pl)</i>
<i>l, ļ</i>	<i>brālis – brāli (Bruder, Pl)</i>
<i>ll, ļļ</i>	<i>rullis – rulli (Rolle, Pl)</i>

In dem selben Schritt wird durch Einführen vom Großbuchstabe T für t, j, s, š auch das Problem der Verben gelöst. Verben bekommen nämlich durch die Konjugation, aber auch in Partizipien und Substantiven oft ein Fugenelement, in anderen Formen ist einer dieser Buchstaben Teil der Flexion. Um die Wörter miteinander vergleichbar zu machen, werden diese 4 Buchstaben zu einem Großbuchstaben zusammengefasst. Zusätzlich wird in Wörtern, die nach dem 5. Schritt auf o, ā, ē, ī, ū enden, ebenfalls T hinzugefügt. Es ist durch den Algorithmus davon auszugehen, dass es sich bei diesen Wörtern ausschließlich um Verben handelt. So werden Verben weitgehend gematcht.

Nicht komplett gelöst werden folgende Fälle des Lautwechsels. Teile davon werden durch andere Regeln abgedeckt (s. Schritt 6 im Algorithmus):

$s - t - z$ *mest – meta – ~~m~~zdam*s (werfen)
 $s - d - z - \check{z}$ *spiest – ~~s~~pieda – ~~s~~piezdam – spie~~z~~u* (drücken)
Nicht gelöst: $g - dz$ *sniegt – ~~s~~niedz*a (reichen)

Nicht gelöst sind alle Probleme, die in der Mitte oder am Anfang des Wortes Probleme bereiten. Das ist ein Problem des Porter Algorithmus, der das Wort nur vom Ende her behandelt.

Vokalischer Lautwechsel:

e $e, \bar{e}$	<i>ņemt – ņēmu</i> (nehmen)
$\bar{a}$ $\bar{a}, a$	<i>kārt – kar</i> (hängen)
au $\bar{a}v$	<i>šaut – šāva</i> (schießen)
ie $ej, \bar{e}j, ien$	<i>iet – iju</i> (gehen), <i>skriet – skrienu – skrēju</i> (laufen)

Negation:

Im Lettischen wird die Negation meistens als Präfix *ne-* angehängt. Da Präfixe im Porter nicht behandelt werden, können die negierten Wörter im Lettischen mit den nicht-negierten nicht gematcht werden.

Das gleiche gilt für den Superlativ (*vis-*) und alle anderen Präfixe.

!!! Selbstverständlich können unregelmäßige Formen nicht gematcht werden. Dafür bräuchte man eine Art Lexikon, was der Porter-Algorithmus nicht vorsieht.

Fehler bei Verben, die in Kauf genommen werden, da entsprechende Regeln zu häufigem overstemming bei Substantiven führen würden:

Die Verbendung *–āt* (2.Pers.Pl.Präs. in einigen Verben und 2.Pers.Pl.Prät. in allen Verben; diese Form ist auch die Höflichkeitsform im Lettischen) wird komplett nicht abgetrennt:

rakstāt (schreiben) – **rakstāT* im Gegensatz zu *rakstīja* – *rakstīT*

runājāt (sprechen) – **runājāT* im Gegensatz zu *runājat* – *runāT*

In Partizipien auf *-am-* und *-ot-*, die aus Verben, die im Präsens auf *-st-* enden, gebildet werden, wird *st* nicht abgetrennt:

mirstama vaina (etw., was zum Tod führt) – **mirsT*, richtig wäre *mir*

mirstot (beim Sterben) – **mirsT*, richtig wäre *mir*

Analyse der Ergebnisse

Ein Korpus aus diversen lettischen Texten von insgesamt **59.803** Wörtern wird zuerst gegen eine Stoppwörterliste (2.153 Wörter) abgeglichen. 12.673 Stoppwörter (23,12%) werden aussortiert. Nach dem Zusammenfassen gleicher Wörter bleiben 9.934 Einträge. Nach dem Stemmen werden diese Einträge auf **4.907** Stämme, also um ca. 50% reduziert. Aus dem Korpus verbleiben 8,2% als Indexterme.

Einige positive Beispiele:

skatoties, skatu, skatāma, skatās -> skaT (sehen)

attīstīt (entwickeln), attīstītas (entwickelt), attīstīties (sich entwickeln), attīstīšanai (Entwicklung)-> attīstīJ

ansambli, ansamblis, ansambļa, ansambļu -> ansambL (Ensemble)

cilvēces (Menschheit), cilvēka, cilvēki, cilvēks, cilvēku (Mensch) -> cilvēC

Beobachtete Probleme:

Es können Fehler entstehen bei Wörtern, die ähnliche Formen haben, aber in ihren Inhalten differieren (s. auch die Analyse des Englischen):

dod (geben), dodas (gehen) -> doZ

datu (Daten), datums (Datum)-> daJ

dome ((Stadt)rat), domām (Gedanken)-> dom

!!! ausu (Ohren), auto (Auto) -> auJ

!!! cietais (hart), cietumos (Gefängnisse), cietusi (gelitten), cieša (eng) -> cieJ

Bei Verben, die auf ein Vokal enden, ist das Resultat nicht zufriedenstellend (dieselben Regeln, die hier Fehler verursachen, sind aber für Substantive unentbehrlich):

akceptē -> akcepT, sollte akceptēT sein

In Partizipien auf *-am-* und *-ot-*, die aus Verbformen auf *-st-* gebildet werden (s. Problembeschreibung vorher):

apgūstamo (das zu erlernende etw.)-> apgūsT, sollte apgūt sein.

Die Verbendung *-āt* 2.Pers.Pl. wird komplett nicht abgetrennt (s. ebenfalls die Problembeschreibung):

redzējāt (sehen) -> redzējāt

Es kommt zu overstemming in einigen Fällen:

darboties (funktionieren) -> *darb*, sollte *darboT* sein

sasitas (zerbrechen) -> *sa* overstemming, sollte *sasiT* sein

plaušu (Lunge) -> *pla*, sollte *plauT* sein

sievietes, *sievietēm* (Frauen) -> *siev*; richtig: *sieviešu* -> *sievieJ*

jāiziet ((2 Präpositionen) jemand muss durch etwas gehen) -> *jāiZ*,
sollte eigentlich zum Verb *izieT* zurückgeführt werden

overstemming, als dessen Folge falsche Ersetzung an der Wortgrenze in einer
Zusammensetzung:

krimināllietā (Strafsache) -> *krimināLL*, sollte *krimināllieT* sein

In dem letzteren Fall wie in vielen anderen ergibt auch die overstemmte
Version immer noch die richtigen matches.

Understemming, da die minimale zu stemmende Wortlänge mit 4
Buchstaben definiert wurde:

būvju (Bauten) *būvT*, sollte *būv* sein

Übersicht der Ergebnisse in Englisch, Französisch, Lettisch

		Medizin			Politik			Wirtschaft		
	Stopp- wörter- liste	Wörter insge- samt	Stopp- wörter aus- sortiert	Terme nach stem- men	Wörter insge- samt	Stopp- wörter aus- sortiert	Terme nach stem- men	Wörter insge- samt	Stopp- wörter aus- sortiert	Terme nach stem- men
Englisch	541	19.992	9.619 48,1	2.716 13,6%	20.482	10.778 52,6%	2.953 14,4%	20.066	9.718 48,4%	2.811 14%
Französisch	171	25.478	10.996 43,2%	3.564 14%	23.224	10.745 46,3%	3.574 15,4%	24.499	10.500 42,9%	905 3,7%
Lettisch	2.153	17.876	6.628 37,1%	3.514 19,7%	25.565	7.116 27,8%	3.437 13,4%	21.412	6.373 29,8%	2.766 12,9%

Auffallend hoch ist der Unterschied in der Anzahl der Stoppwörter. Nach dem Aussortieren der Stoppwörter reduzieren sich Korpora um fast die Hälfte. Die Ergebnisse nach dem Stemmen halten sich in etwa die Waage. Nur der lettische Korpus im Bereich Politik divergiert mit 27,8% und der französische Korpus im Bereich Wirtschaft mit 3,7% erheblich. Dies mag an der Auswahl der jeweiligen Texte liegen und bedürfte noch genauerer Prüfung, was an dieser Stelle aber zu weit führen würde.

Nice Stemmer

Entwickelt 1996 von

- Kiduk Yang
- Denqi Song
- Wooseob Jeoung
- Rong Tang

Nice Stemmer

Nice Stemmer ist ein kombinierter Stemmer
aus:

Simple Stemmer

Porter Stemmer

Inflection Stemmer (Krovetz)

Nice Stemming: Simple Stemmer

Einfache Entfernung (Abschneiden) von Pluralsuffixen (Englisch nur ,s‘)

Ignorieren von unregelmäßigen Pluralformen,
Tempora

Nice Stemming: Porter Stemmer

Simple Pluralentfernung

Morphologische Variationen

Schwachstellen:

- Irreguläre Pluralformen
- Ignoriert Tempora
- Vereinzelte Suffixabtrennungen funktionieren nicht.

Nice Stemming: Inflection Stemmer (Krovetz)

Suche im Wörterbuch nach dem Wort

Nicht gefunden: ersetze Suffixe ,ing‘, ,es‘, und ,ed‘ durch e

Erneute Suche des neu erstellten Terms

Nicht gefunden: e wird entfernt und erneute Suche

JETZT: interessante Laufzeiten!

Best case: $O(n)$

Worst case: $O(3n)$

Nice Stemming: Kombiniertes Stemming

Wort für Wort simultan nach den 3 vorherigen Methoden stemmen

Überprüfung des Wörterbuches, welche Wörter enthalten sind

Falls alle gestemmtten Wörter gefunden wurden, nimm das kürzeste

Falls eines der gestemmtten Wörter gefunden wurde, benutze es

Aber: Irreguläre Formen und Tempora werden nach wie vor ignoriert

Quellen

- Frakes, William.B. (1992): *Stemming Algorithms*. In: Information Retrieval. Data Structures & Algorithms. William B. Frakes & Ricardo Baeza-Yates (eds.). PTR Prentice-Hall: Englewood Cliffs, New Jersey. S. 131-160.
- Kowalski, Gerald J. & Maybury, Mark T. (eds.) (2000): *Information Storage and Retrieval Systems. Theory and Implementation*. 2nd ed. Kluwer Academic Publishers: Boston u.a.
- Porter-Algorithmus und Implementierung in Snowball für zahlreiche Sprachen
www.snowball.tartarus.org
- Porter-Algorithmus und Implementierung in verschiedenen Programmiersprachen fürs Englische
www.tartarus.org/~martin/PorterStemmer/
- Porter-Algorithmus und Implementierung in verschiedenen Programmiersprachen fürs Französische
snowball.tartarus.org/french/stemmer.html
- Rossen Kowatschew, Johannes Lang (2002): Stemming. Seminarreferat
www.rzuser.uni-heidelberg.de/~jlang/Stemming/
- Kiduk Yang, Denqi Song, Wooseob Jeong, Rong Tang: Nice Stemmer. Projekt
www.ils.unc.edu/yangk/nstem.htm